Klaus x DevOps

Anders Mouanga `mouanga@kth.se`
Erik Söderlund `eriso@kth.se`

October 11, 2026

1 Introduction

This project adds several DevOps features to a fork of Jonas Haag's `klaus` project (https://github.com/jonashaag/klaus). `klaus` is a web-based Git viewer that is designed to be easy to set up and use. It can be run using Docker, pip, or a multitude of other ways. To narrow down the scope of this project, we have decided to limit our version of `klaus`,appropriately called `klaus-clone`, and hosted on GitHub (https://github.com/superderker/klaus-clone) ,to run on Docker only.

2 Features

2.1 Continuous integration

We do not push changes directly to the main branch. Instead, we use the regular system of pull requests. Any time a pull request is made, we use a GitHub Actions workflow to run the following:

- Ruff: a linter that verifies that the proposed code maintains the proper style.
- mypy: a tool that looks at the Python code's type annotations and makes sure it all makes sense.
- Bandit: another linter, scans the code for security vulnerabilities.
- Gitleaks: scans pull requests to see if any secrets (like API tokens) have been leaked.

These tools are used as they are fast (especially Ruff), industry-standard DevOps software that are suitable for Python development. Together, they cover code style, correctness and safety. Furthermore, we use a subset of the tests from the original `klaus`.

2.2 Infrastructure and Terraform

Terraform is used to automate the provisioning of the infrastructure. It spins up an AWS Ubuntu 22.04 virtual server and automatically installs Docker on it, allowing it to host Docker containers.

2.3 Continuous deployment

CD builds, pushes, scans with Trivy (fails on fixable CRITICAL or HIGH vulnerabilities), then deploys over SSH with git pull, docker compose pull and docker compose up -d, using GitHub secrets.

2.4 Runtime configuration

Docker Compose defines two containers on a private network: Klaus and an Nginx reverse proxy that is the only service exposed to the internet. The credentials file is provided as a Docker secret, and the repositories are mounted read-only.

2.5 Secrets

No secrets are stored in the repository or in the Docker image. The authentication file for Klaus (`htdigest.txt`) is created by hand on the server(aws hosted version) and mounted into the container as a Docker secret, so it is only visible to the application at runtime. It is excluded from Git and from the Docker build context. The deployment credentials, the server address and the SSH private key, are stored as encrypted GitHub Actions secrets and are only available to the CD workflow, while publishing to the container registry uses the temporary token that GitHub generates for each run.

2.6 Dependabot

In addition, Dependabot is used to monitor software packages (in this case Python packages) for security vulnerabilities and outdated versions. If Dependabot finds an outdated or

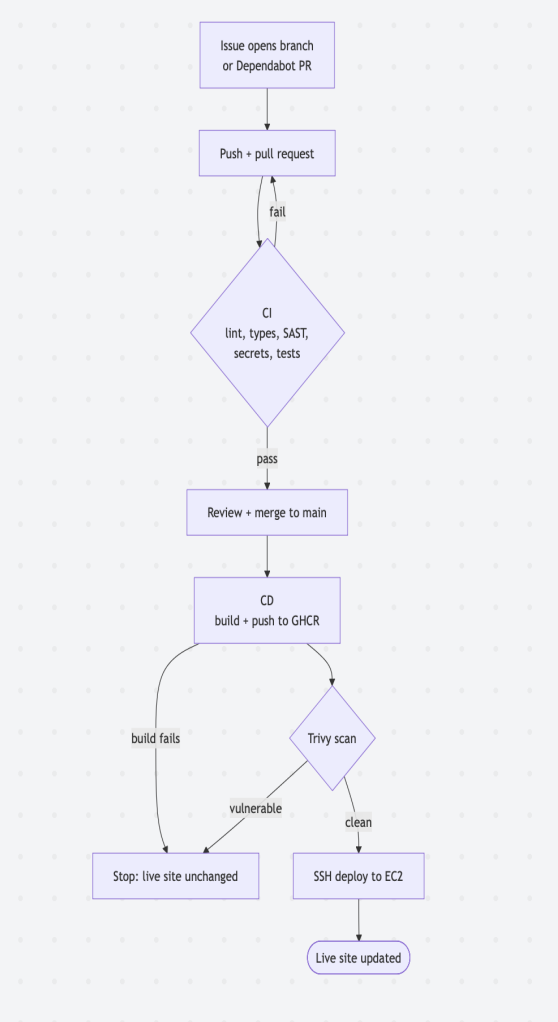


Figure 1: The pipeline from pull request to live deployment.

insecure package, it automatically opens a pull request and tries to fix the issue without breaking existing code.

2.7 CodeRabbit

CodeRabbit runs next to the CI pipeline as an additional verification step to ensure code quality. It scans pull requests and generates comments about the code.

3 The full DevOps pipeline

Any code, whether it be Terraform code for IaC or ordinary Python code, is subject to the CI/CD pipeline. The same goes for changes made by Dependabot. Before any code is merged into the main branch, it is run through the CI pipeline described above. Then, when it is accepted, it is automatically deployed to a Docker container. The Docker container is then deployed to the AWS instance controlled by the Terraform code. In the meantime, Dependabot runs its weekly checks. The diagram below aids to further illustrate the pipeline step by step.

4 Critical evaluation

A limitation of our DevOps pipeline is the tooling coverage; it does not catch all bugs or issues (and by Rice's theorem, doing so is even impossible!). This could have been mitigated by adding even more tools, but this could lead to notification blindness, longer development time, and a project with a larger scope than intended. All updates cause some downtime because only one server is in use. Using e.g. blue-green deployment would have introduced unnecessary complexity (and cost) to a project of this size. Furthermore, having only one server introduces a single point of failure and so the redundancy is very low. The decision to do so was made based on its lower complexity and cost.

5 Use of AI tools

Artificial intelligence was used as part of the CI pipeline through the use of CodeRabbit. Furthermore, it was also consulted for help with ideas for the code, but the implementation and final product is ours.